

Day 3

Python and Object-Oriented Programming

Some Key Ideas

- **Class** – A class is a blueprint for creating objects. It defines the properties (variables) and methods (functions) that its objects will have.
 - **Object** – An object is an instance of a class. It holds specific values for the properties defined by the class and can use its methods.
 - **Properties** – Pieces of data that are associated with an object (nouns/adjectives).
 - **Methods** – Functions and procedures that an object can run (verbs).
-

The Point of OOP

The point of object-oriented programming (OOP) is that it helps us model real-world things in our code. When we're programming a robot, it's helpful for our code to reflect the parts and systems of the actual robot we're building and controlling.

For example, we can create a class to represent a motor, then make multiple motor objects, each representing a specific part of the robot. Each motor can have its own speed and methods to control it, but they all share the same structure defined by the class.

Creating Classes

You create a class using this format:

```
class MyClass:
    def __init__(self, args, more_args):
        self.property = args

    def my_cool_method(self, args, more_args):
        # my code
        pass
```

In this code:

- The class is called `MyClass`.
- Each object made from this class will have its own version of the property called `property`.
- The method `my_cool_method()` can be used by any object of this type.

Example:

```
my_object = MyClass("value1", "value2") # Create an object of MyClass
my_object.my_cool_method()              # Call its method
```

The `__init__` Method

In Python, whenever an object is instantiated (i.e., created), it runs a special method called `__init__`. This is also known as the **constructor**. It allows us to set up the object with its starting properties.

The `self` Keyword

The `self` keyword is a reference to the object that is calling the method. It lets us access and update that object's properties inside the class. Every method in a class must include `self` as its first parameter, and any property should be defined using `self`, like:

```
self.speed = 0.5
```

This ensures the property belongs to the specific object and is available throughout all its methods.

Abstraction and Libraries

One of the benefits of OOP is **abstraction** — the idea that we can use code written by others without needing to understand how it works inside. For example, CTRE (Cross The Road Electronics) makes motor controllers like the TalonFX. They also provide classes that let us use these devices in our code.

If we read the CTRE API documentation, we see that a TalonFX object has a method called `set()` that takes a number from -1 to 1 to control the motor speed.

```
from phoenix5 import TalonFX, ControlMode

my_motor = TalonFX(3) # Motor connected to CAN ID 3
my_motor.set(ControlMode.PercentOutput, 1.0) # Run the motor at full forward
```

So when you're programming a robot, a big part of the job is learning the classes and methods provided by libraries like WPILib and CTRE, not writing everything from scratch.

Practice Time

Let's put this into practice. Choose one task based on your comfort level:

Beginner

- Create a new project with a file that defines a class called `Motor`.
- The motor should have a property called `speed` set to 0.5 when it's created.
- Add a method called `set_speed()` that changes the speed.
- Add two more methods: `speed_up()` (doubles the current speed) and `slow_down()` (halves the speed).

Intermediate

- Create two classes: `Motor` (as above) and `MotorControllerGroup`.
- A `MotorControllerGroup` should:
 - Be initialized with a list of motors.
 - Have an `add_motor()` method that adds another motor to the group.
 - Have a `set_speed()` method that sets the speed of **all** the motors in the group.

Advanced

- Research the difference between `@classmethod` and `@staticmethod`.
 - Write a short explanation in your own words of the difference.
 - Come up with a potential use case for each in the context of programming a robot.
-

Bonus Tip

Want to print your motor objects in a readable way? Try defining a `__repr__()` method in your class:

```
def __repr__(self):
    return f"<Motor speed={self.speed}>"
```

This helps with debugging and lets you quickly see the state of your motors in the console.
